

Database Map

تعليمات حاسمة لفهم قاعدة البيانات قبل أي تطوير أو استكمال

هذا الجزء **واحد من أهم الأجزاء على الإطلاق** لفهم منصة **نَسَق | NASSAQ**. لا تبدأ أي تطوير حقيقي، ولا أي إعادة هيكلة، ولا أي إضافة Features، ولا أي تنظيف بيانات، ولا أي بناء محركات أو صفحات جديدة، قبل أن تقوم بإنتاج **Database Map شامل ودقيق ومفسر**.

يجب أن تفهم أن **قاعدة البيانات في نَسَق ليست طبقة تخزين فقط**، بل هي:

- مصدر الحقيقة الوحيد للنظام
- المرجع الأساسي لكل Dashboard و Card و Chart و Counter
- الأساس الذي تبنى عليه العلاقات بين المدارس والمستخدمين والأدوار
- المصدر الذي تعتمد عليه جميع المحركات
- الطبقة التي تحدد ما إذا كان النظام يعمل ك **School Operating System حقيقي** أو مجرد واجهات شكلية

أي خطأ في فهم قاعدة البيانات سيؤدي إلى:

- بناء منطق خاطئ
- كسر العلاقات بين الكيانات
- إظهار بيانات غير صحيحة
- إنشاء Features تعمل ظاهريًا لكنها غير متماسكة
- إفساد الـ multi-tenant isolation
- إفساد الـ unified identity model
- إفساد التقارير والجداول والحضور والصلاحيات

لهذا السبب، المطلوب منك هنا ليس مجرد "قراءة schema"، بل إنتاج **خريطة تشغيلية كاملة لقاعدة البيانات**.

الهدف من Database Map

الهدف من هذا الجزء هو أن تنتج **خريطة بنيوية + تشغيلية + منطقية** لقاعدة البيانات، بحيث نستطيع من خلالها معرفة:

- ما هي الكيانات الأساسية التي يقوم عليها النظام
- ما هي الجداول المساندة
- ما هي الجداول المرجعية
- ما هي الجداول التشغيلية اليومية
- ما هي الجداول التي تمثل قواعد عمل النظام
- ما هي الجداول التي تمثل output أو data مشتقة
- كيف ترتبط الجداول ببعضها
- ما هو مسار البيانات من الإنشاء حتى الظهور في الواجهات والتقارير
- ما الذي يمكن تعديله بأمان
- ما الذي لا يجب المساس به إلا بحذر شديد
- ما الذي يمكن توليده تلقائيًا
- وما الذي يمثل أصلًا لا يجوز استبداله أو حذفه

المطلوب منك بالتفصيل الدقيق

(1) حصر جميع الجداول الموجودة فعليًا

ابدأ بعمل **Inventory كامل** لجميع الجداول الموجودة داخل قاعدة البيانات الفعلية، أو المعرفة داخل:

- schema
- ORM models
- migration files
- SQL definitions
- seed files إن كانت تنشئ جداول أو علاقات ضمنيًا

المطلوب هنا:

لكل جدول، اذكر:

- الاسم الفعلي للجدول كما هو موجود
- إن كان له اسم بديل أو alias مستخدم في الكود
- هل الجدول مستخدم فعليًا أم قديم / legacy / غير مستخدم
- هل الجدول أساسي أم ثانوي
- هل الجدول مرتبط مباشرة بواجهات النظام أم لا

مهم جدًا:

لا تعتمد على اسم الجدول فقط لتخمين وظيفته.
يجب التحقق من:

- الأعمدة
- العلاقات
- usage داخل services
- usage داخل queries
- usage داخل APIs
- usage داخل التقارير

لأن بعض الجداول قد تبدو من اسمها بسيطة، لكنها في الحقيقة جوهريّة جدًا، والعكس صحيح.

(2) شرح وظيفة كل جدول بدقة تشغيلية

لكل جدول، لا يكفي أن تقول "هذا جدول الطلاب" أو "هذا جدول المدارس".
المطلوب شرح **وظيفته التشغيلية الدقيقة** داخل النظام.

لكل جدول وضح:

- ماذا يمثل هذا الجدول داخل نَسَق؟
- هل يمثل كيانًا أساسيًا أم علاقة بين كيانيين أم سجل حدث أم إعدادًا أم قيمة مرجعية؟
- ما هي مسؤوليته داخل منطق النظام؟
- ما الذي يعتمد عليه؟
- ما الذي يعتمد هو عليه؟

- هل يُقرأ فقط أم يُقرأ ويُكتب باستمرار؟
- هل يؤثر على واجهات مرئية مباشرة؟
- هل يؤثر على تقارير / auth / permissions / counters / dashboards؟

أمثلة على مستوى التفسير المطلوب:

بدل أن تقول:

جدول schools لتخزين المدارس

يجب أن تشرح مثلاً:

جدول schools يمثل الـ tenant الأساسي داخل النظام. كل مدرسة تعتبر كياناً تشغيلياً مستقلاً. هذا الجدول هو نقطة الارتباط الرئيسية بين الإدارة المركزية وبيانات المدرسة. جميع الكيانات المدرسية مثل الطلاب والمعلمين والفصول والجدول والإعدادات يجب أن ترتبط به بشكل مباشر أو غير مباشر. أي خلل في هذا الجدول أو علاقاته قد يؤدي إلى كسر tenant isolation بالكامل.

هذا هو مستوى العمق المطلوب.

3) تصنيف الجداول إلى فئات واضحة

يجب أن تقوم بتقسيم جميع الجداول إلى تصنيف بنوي واضح، وليس عرضها كقائمة مسطحة فقط.

صنف الجداول إلى الفئات التالية على الأقل:

أ) Core Entity Tables

الجدول التي تمثل الكيانات الجوهرية الأساسية للنظام

ب) Relationship / Junction Tables

الجدول التي تنشئ الربط بين الكيانات

ج) Reference Tables

الجدول المرجعية التي تحتوي على قيم ثابتة أو شبه ثابتة يعتمد عليها النظام

د) Operational Tables

الجدول التي تمثل العمليات اليومية الفعلية للنظام

هـ) Settings / Configuration Tables

جداول الإعدادات العامة أو الخاصة بالمدرسة أو المستخدم

و) Security / Auth Tables

الجدول المتعلقة بالمصادقة والجلسات والصلاحيات والأمان

ز) Audit / Log Tables

الجدول التي تسجل النشاطات والأحداث والسجلات

الجدول أو الـ views أو البنى التي تحتوي على بيانات مشتقة أو نتائج مجمعة

أي جدول قديم أو غير واضح أو يبدو أنه لم يعد مستخدمًا أو قد يسبب تضاربًا

الهدف من التصنيف

هذا التصنيف مهم جدًا لأنه سيحدد:

- ما الذي لا يمكن العبث به
- ما الذي يمكن refactor له
- ما الذي يمكن تنظيفه
- ما الذي يجب ربط features الجديدة به
- ما الذي يمكن أن يكون مصدرًا للأخطاء أو التضارب

4) تحديد الـ Core Entities بدقة

أنت تحتاج إلى تحديد الكيانات الأساسية الحاكمة للنظام. هذه ليست مجرد جداول مهمة، بل هي الجداول التي يقوم عليها منطق نسق بالكامل.

غالبًا، الـ Core Entities في نسق تشمل:

- المدارس
- الهويات الموحدة للمستخدمين
- الحسابات / المستخدمين
- الأدوار
- ربط الأدوار
- الطلاب
- المعلمين
- أولياء الأمور
- الفصول
- الصفوف الدراسية
- المواد
- الجداول
- الحصة / schedule entries
- الحضور
- التقييمات
- الإشعارات
- الإعدادات
- الطلبات
- سجلات التدقيق

المطلوب:

لكل Core Entity:

- اشرح لماذا هو Core
- ما الذي يبنى عليه
- ما الذي ينهار إذا اختل
- هل له امتداد على مستوى School أم Platform أم الاثنين

مهم:

لا تخط بين:

- الكيان الأساسي
- والجدول المشتق
- والجدول الرابط
- والجدول المرجعي

(5) تحديد الجداول المرجعية بدقة

الجدول المرجعية جزء بالغ الحساسية. غالبًا هي لا تتغير كثيرًا، لكنها تؤثر على النظام بأكمله.

أمثلة متوقعة:

- أنواع الأدوار
- حالات الحسابات
- المراحل الدراسية
- الصفوف
- المواد الأساسية
- أنواع الإشعارات
- حالات الحضور
- أنواع التقييمات
- أنواع الطلبات
- أنواع الجلسات أو السجلات أو الفئات

المطلوب:

لكل جدول مرجعي:

- اشرح وظيفته
- هل هو global أم tenant-specific
- هل النظام يعتمد عليه في validations
- هل النظام يعتمد عليه في UI dropdowns / selectors
- هل تغييره قد يؤثر على السلوك أو التقارير أو الصلاحيات
- هل يجب seed له دائمًا
- هل يمكن حذفه أو تعديل قيمه أم لا

قاعدة مهمة:

أي جدول مرجعي لا يجب التعامل معه كبيانات عادية. بل غالبًا يجب اعتباره جزءًا من "قواعد تشغيل المنصة".

6) تحديد الجداول التشغيلية اليومية

هذه الجداول تمثل "نبض النظام" اليومي. وهي الجداول التي تُنشأ وتُحدَّث باستمرار أثناء تشغيل المدارس.

أمثلة:

- الطلاب
- حضور الطلاب
- حضور المعلمين
- الجداول
- الحصص
- التقييمات
- الرسائل
- الإشعارات
- السجلات
- الطلبات
- نقل الطلاب
- إسنادات المعلمين
- تحديثات الإعدادات
- محاولات الدخول

المطلوب:

لكل جدول تشغيلي:

- اشرح متى يُنشأ السجل فيه
- من الذي ينشئه
- أي Role أو صفحة أو Service مسؤولة عن الكتابة فيه
- ما هي الصفحات أو التقارير أو المحركات التي تقرأ منه
- هل له lifecycle واضح
- هل يمكن أرشفته
- هل يجب أن يظهر مباشرة في الواجهة
- هل يجب أن يُسجل تغييره في audit logs

7) تحديد الجداول المشتقة Derived Data بوضوح شديد

هذه نقطة مهمة جدًا لأن كثيرًا من الأنظمة تختلط فيها البيانات الأصلية مع البيانات المشتقة.

يجب أن تميز بوضوح بين:

البيانات الأصلية المصدرية

مثل:

- الطالب
- الفصل
- الحضور

- التقييم
- المدرسة

البيانات المشتقة أو المحسوبة

مثل:

- عدد الطلاب
- نسبة الحضور
- متوسط الدرجات
- KPI cards
- chart datasets
- dashboard summaries
- report aggregates
- monthly trends
- comparative metrics

المطلوب:

- حدد هل توجد جداول مخصصة لحفظ بيانات مشتقة
- أم أنها تحسب عند الطلب
- أم يوجد materialized views أو snapshots أو cached summaries
- أم توجد طبقة تقارير تولدها من source tables

ثم وضح:

- ما الذي يجب اعتباره مصدر الحقيقة
- ما الذي هو مجرد output
- ما الذي يمكن إعادة توليده من source data
- ما الذي لا يجب تعديله يدويًا أبدًا

قاعدة صارمة:

لا تسمح أبدًا بأن يتم التعامل مع derived data على أنها أصل النظام.
الأصل دائمًا هو الجداول المصدريّة.

8) شرح العلاقات بين الجداول بشكل معماري لا سطحي

هذا من أهم الأجزاء.
نريد منك أن ترسم أو تشرح **العلاقات الحقيقية** بين الجداول، لا مجرد ذكر foreign keys.

المطلوب:

لكل علاقة مهمة، وضح:

- نوع العلاقة
- اتجاه العلاقة
- لماذا هذه العلاقة موجودة
- كيف يستخدمها النظام

- ما الذي ينتج عن كسرها
- هل هي mandatory أم optional
- هل تتكرر عبر tenant boundaries أم معزولة داخل tenant

العلاقات التي يجب شرحها على الأقل:

- علاقة المدرسة بالمستخدمين
- علاقة User Identity بالأدوار
- علاقة المستخدم بالدور النشط
- علاقة الطالب بولي الأمر
- علاقة الطالب بالفصل
- علاقة المعلم بالمدرسة
- علاقة المعلم بالفصول
- علاقة المعلم بالمواد
- علاقة الفصل بالصف والمرحلة
- علاقة الجدول بالفصل
- علاقة الجدول بالمعلم
- علاقة الجدول بالحصّة
- علاقة الحضور بالحصّة
- علاقة التقييم بالطالب / الفصل / المادة
- علاقة الإشعارات بالمستلمين
- علاقة الطلبات بالمستخدمين أو المدارس

ما نريده هنا

نريد أن نفهم:

- كيف يتحرك النظام
- كيف تنتقل البيانات بين المحركات
- وما هي نقاط الربط الجوهرية

(9) توثيق المفاتيح الأساسية والخارجية والقيود

لكل جدول، وثّق بدقة:

Primary Key (أ)

- اسمه
- نوعه
- هل هو UUID أم integer أم غير ذلك
- هل هو surrogate key أم طبيعي

Foreign Keys (ب)

- ما هي الجداول التي يشير إليها
- ما نوع العلاقة
- هل يوجد cascading أو soft delete strategy
- هل هناك FK منطقي في الكود لكنه غير مفروض DB-level

Unique Constraints (ج)

مثل:

- البريد الإلكتروني
- رقم الهوية
- رقم الطالب
- رمز المدرسة
- username
- tenant composite uniqueness داخل

Nullability / Required Fields (د)

- ما الحقول الإلزامية
- ما الحقول الاختيارية
- ما الحقول التي إذا تُركت فارغة تتسبب في كسر feature معينة

Status Fields / Soft Delete / Flags (هـ)

- هل يوجد is_active
- هل يوجد is_deleted
- هل يوجد status
- هل يوجد archived_at
- هل يوجد suspended_at

المطلوب:

اشرح كيف تؤثر هذه الحقول والقيود على:

- business logic
- visibility في الواجهة
- reports
- permission checks
- search / filters
- lifecycle للكيان

10) تحديد الجداول التي لا يجب حذفها أو العبث بها

هذه نقطة بالغة الأهمية.

يجب أن تحدد بوضوح شديد الجداول أو السجلات أو الكيانات التي لا يجوز حذفها أو تعديلها بعشوائية لأنها جزء من تشغيل النظام أو أمانه أو هويته المعمارية.

أمثلة محتملة:

- جداول الأدوار الأساسية
- جداول الصلاحيات
- جداول الهوية الموحدة
- إعدادات النظام العامة

- إعدادات tenant / المدرسة
- الحسابات الرسمية الأساسية
- جداول auth والجلسات
- الجداول المرجعية الأساسية
- جداول الربط المحورية
- الجداول التي يعتمد عليها tenant isolation
- الجداول التي يعتمد عليها role switching
- السجلات القانونية أو الإصدارات المعتمدة للشروط والسياسات

المطلوب:

- حددها بالاسم
- اشرح لماذا لا يجب حذفها
- ما الذي قد ينكسر إذا تم حذفها أو تغييرها
- هل يمكن فقط تعطيلها بدل حذفها
- هل يجب التعامل معها عبر soft delete فقط

11) تحديد ما الذي يمكن توليده أو Seed له

في المقابل، نريد منك تحديد ما هي الكيانات أو الجداول التي يمكن:

- seed لها
- توليدها تلقائيًا
- إعادة إنشائها
- استخدامها لبيانات تشغيلية واقعية
- أو تنظيفها وإعادة بنائها إذا لزم

أمثلة محتملة:

- بيانات تشغيل اختبارية
- طلاب ومعلمون تجريبيون
- جداول دراسية توليدية
- إشعارات تشغيلية
- logs يمكن تدويرها
- aggregated metrics يمكن إعادة حسابها
- dashboard summaries يمكن إعادة بنائها
- attendance test records
- mock-independent operational data إذا كان الهدف اختبار النظام

لكن:

يجب أن تميز بوضوح بين:

- ما يمكن توليده بأمان
- وما يجب أن يكون ناتجًا عن تشغيل حقيقي
- وما يجب الحفاظ عليه دائمًا

12) تحديد تأثير كل جدول على الواجهات والمحركات

لكل جدول أو مجموعة جداول، وضح:

- أي صفحة تعتمد عليه
- أي كارت أو dashboard يقرأ منه
- أي محرك يستخدمه
- أي API يعتمد عليه
- هل يؤثر على auth
- هل يؤثر على schedule
- هل يؤثر على attendance
- هل يؤثر على notifications
- هل يؤثر على reporting
- هل يؤثر على tenant boundaries

الهدف

نريد منك إنتاج **Dependency Awareness Map** وليس فقط Database Map.

أي أن نفهم:

- هذا الجدول إذا تغير، ما الذي سيتأثر؟
- هذا الـ FK إذا انكسر، أي Features ستتهرب؟
- هذا الجدول إذا مُسح، هل تختفي cards فقط أم ينهار auth أم ينهار scheduling؟

13) اكتشاف الجداول القديمة / المشكوك فيها / الزائدة

أثناء التحليل، يجب أن تضع قائمة خاصة بـ:

- الجداول القديمة
- الجداول غير المستخدمة
- الجداول التي يبدو أنها legacy
- الجداول المكررة وظيفيًا
- الجداول التي قد تكون remnants من prototype أو demo
- الجداول التي لا تتسق مع architecture الحالية

المطلوب:

لكل جدول من هذا النوع، وضح:

- لماذا تعتقد أنه قديم أو غير مستخدم
- هل ما زال له references في الكود
- هل يُقرأ في reports أو APIs
- هل يمكن حذفه أم يجب أولاً عزله
- هل ينبغي تحليله أكثر قبل اتخاذ قرار

مهم جدًا:

لا تتخذ قرار حذف فقط لأن الجدول "يبدو غير مستخدم".
بل وثّق أولاً، ثم صعد الملاحظة للمراجعة.

14) ربط قاعدة البيانات بالمفاهيم الكبرى للنظام

نريد منك أن تشرح كيف تمثل قاعدة البيانات فعليًا المفاهيم الجوهرية التالية:

أ) Multi-Tenant

- أين يتم تمثيل tenant / school context؟
- كيف يتم عزل المدارس؟
- هل جميع الجداول التشغيلية مرتبطة بمدرسة؟
- هل توجد جداول global لا ترتبط بمدرسة؟
- كيف يتم حماية الـ cross-tenant leakage؟

ب) Unified User Identity

- أين توجد الهوية الموحدة؟
- أين تُربط الأدوار؟
- كيف يُدار switch role؟
- كيف يُمنع تكرار نفس الدور؟
- كيف يتم تمثيل العلاقة بين role و account و identity؟

ج) Platform Level vs School Level

- أي الجداول platform-wide؟
- أي الجداول tenant-specific؟
- أي الجداول مختلطة؟
- ما هي الجداول التي يجب أن تتغير queries الخاصة بها حسب السياق؟

د) Source of Truth

- ما هي الجداول المصدريّة لكل feature؟
- من أين يجب أن تأتي counters؟
- من أين يجب أن تأتي charts؟
- من أين يجب أن تأتي dashboard KPIs؟
- ما هي الجداول التي لا يجوز استبدالها بـ frontend values؟

15) شكل المخرجات المطلوب منك

أريد منك أن تنتج هذا الجزء في صورة منظمة جدًا، تشمل على الأقل:

1. قائمة الجداول الكاملة

مع التصنيف

2. جدول لكل كيان

يتضمن:

- الاسم
- النوع

- الوظيفة
- هل هو core / reference / operational / derived
- هل هو global أو tenant-specific
- هل هو حساس
- هل يمكن توليده
- هل لا يجب حذفه

3. وصف العلاقات

إما:

- ERD
- أو pseudo-ERD
- أو textual relationship map منظم جدًا

4. قائمة بالمفاتيح والقيود

لكل جدول

5. قائمة بالـ dangerous tables / protected tables

أي الجداول أو السجلات التي لا يجب لمسها

6. قائمة بالـ derived / recalculable structures

ما الذي يمكن إعادة توليده

7. قائمة بالجدول legacy أو المشكوك فيها

مع توصية تحليلية وليس قرارًا متسرعًا

8. ملخص تنفيذي

يشرح:

- العمود الفقري للنظام
- أين توجد أخطر العلاقات
- أين توجد احتمالات الكسر
- وأين يجب الحذر أثناء التطوير

تعليمات صارمة جدًا أثناء تنفيذ هذا الجزء

- لا تفترض وظيفة جدول من اسمه فقط
- لا تخلط بين الكيان والـ relationship table
- لا تخلط بين source data و derived data
- لا تعتبر أي جدول غير مهم لمجرد أنه لا يظهر مباشرة في UI
- لا تحذف أو توصي بالحذف قبل التأكد من references
- لا تتعامل مع الجداول المرجعية كأنها بيانات عادية
- لا تعتبر counters أو charts مصدرًا للحقيقة

- لا تنسَ tenant isolation في أي تحليل
- لا تنسَ unified identity architecture
- لا تنسَ أن النظام يعتمد على database-first truth

النتيجة التي نريدها منك

نريد منك أن تصل إلى مستوى يكون فيه واضحًا تمامًا:

- كيف يعيش النظام داخل قاعدة البيانات
- كيف ترتبط المدارس بالمستخدمين
- كيف تتحرك الأدوار
- كيف تبني الجداول والحضور والتقييمات
- كيف تنتج التقارير
- ما الذي يمكن تطويره بأمان
- وما الذي سيكسر المنصة إذا تم لمسه بدون فهم

الخلاصة الحاسمة

تعامل مع Database Map على أنه:

- خريطة النظام الحقيقية
- الأساس الذي ستبني فوقه كل شيء
- المرجع الذي يمنعك من اختراع منطق غير موجود
- الوثيقة التي ستحدد هل ستفهم نَسَق كمنصة تشغيل مدرسية حقيقية أم ستتعامل معه كمجموعة صفحات منفصلة

إذا لم تفهم قاعدة البيانات بعمق، فلن تفهم نَسَق فعلاً.